Tiendas 3B · Subdirección de Inteligencia Artificial

Guía paso a paso: inicio de sesión y roles con Microsoft Entra ID

Para una app React (PWA) y un backend FastAPI. Versión «for dummies»: no se asume experiencia previa con Entra.

Qué vas a lograr

- Que el coordinador inicie sesión con su cuenta corporativa desde el teléfono.
- Que tu API sepa **quién** es (el identificador **oid**) y **qué rol** tiene (Coordinador o Admin).
- Que ese **oid** sea el que filtra los registros que cada coordinador puede ver.

Qué sí y qué no incluye esta guía

- **Las imágenes son esquemas, no capturas reales.** No tengo acceso al portal de Entra de Tiendas 3B. Cada esquema muestra la ruta de menú y los campos a llenar. El portal puede verse distinto o estar en español: los nombres en inglés son los de la documentación de Microsoft.
- **El código de FastAPI sí lo probé** localmente con tokens de prueba firmados por mí: sin token, token inválido, expirado, audiencia incorrecta, y rol correcto e incorrecto.
- **El código de React no lo ejecuté** contra tu tenant. Es la configuración estándar de MSAL React y se valida en la Parte D.
- Fuera de alcance: acceso condicional/Intune, identidad administrada y notificaciones en Teams (se piden a identidad en la sección 8).

Contenido

Sección	Qué contiene
1	Conceptos mínimos (glosario sin tecnicismos)
2	Panorama: qué se va a crear y cómo viaja el token
3	Antes de empezar: permisos y hoja de datos a anotar
4	Parte A · Configuración en Entra (10 pasos)
5	Parte B · Frontend React con MSAL
6	Parte C · Backend FastAPI (probado)
7	Parte D · Cómo probar que todo funciona
8	Errores comunes, qué pedir al equipo de identidad y fuentes

1. Conceptos mínimos

Entra ID es el «portería» corporativo: verifica quién eres y te entrega un gafete digital. Tu app y tu API nunca ven la contraseña; solo revisan el gafete.

Término	En palabras simples	Dónde lo verás
Tenant	La «instancia» de Entra de Tiendas 3B. Tiene un identificador único (Tenant ID).	Overview de cualquier registro
App registration (registro de aplicación)	El «alta» de tu app en Entra. Sin registro, Entra no sabe que existe y no te deja iniciar sesión.	Entra ID > App registrations
Client ID (Application ID)	El número de identificación de cada registro. No es un secreto.	Overview del registro
Redirect URI	La dirección a la que Entra te regresa después de iniciar sesión. Debe coincidir letra por letra.	Authentication del SPA
Scope (permiso delegado)	El «permiso» que pide tu app para llamar a tu API a nombre del usuario. Aquí: access_as_user.	Expose an API
App role (rol de aplicación)	Una etiqueta que se le pone al usuario (Coordinador o Admin). Aparece dentro del token como **roles**.	App roles
Token de acceso (JWT)	El gafete digital. Es un texto firmado por Entra con datos del usuario. Dura poco tiempo.	Lo recibe el frontend
oid	El identificador único e inmutable del usuario en tu tenant. Es la llave para filtrar sus registros.	Dentro del token
MSAL	La librería de Microsoft que hace el inicio de sesión y pide los tokens por ti.	@azure/msal-react
PKCE	Un candado extra del flujo de inicio de sesión para apps de navegador. MSAL lo hace solo.	No tienes que programarlo

2. Panorama

Así viaja el inicio de sesión. El frontend nunca habla con la base de datos: siempre pasa por la API, y la API siempre revisa el token.

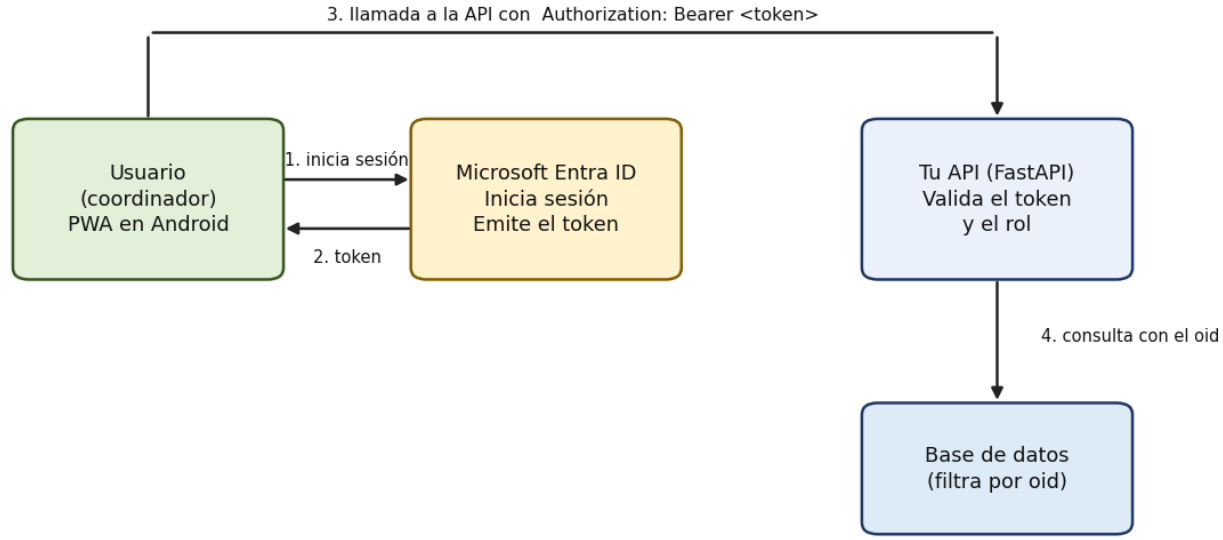


Figura 1. Flujo de inicio de sesión y llamada a la API.

Necesitas **tres registros** en Entra: uno para la API y uno para cada frontend. Los roles se definen en la API, y los usuarios se asignan a esos roles mediante grupos.

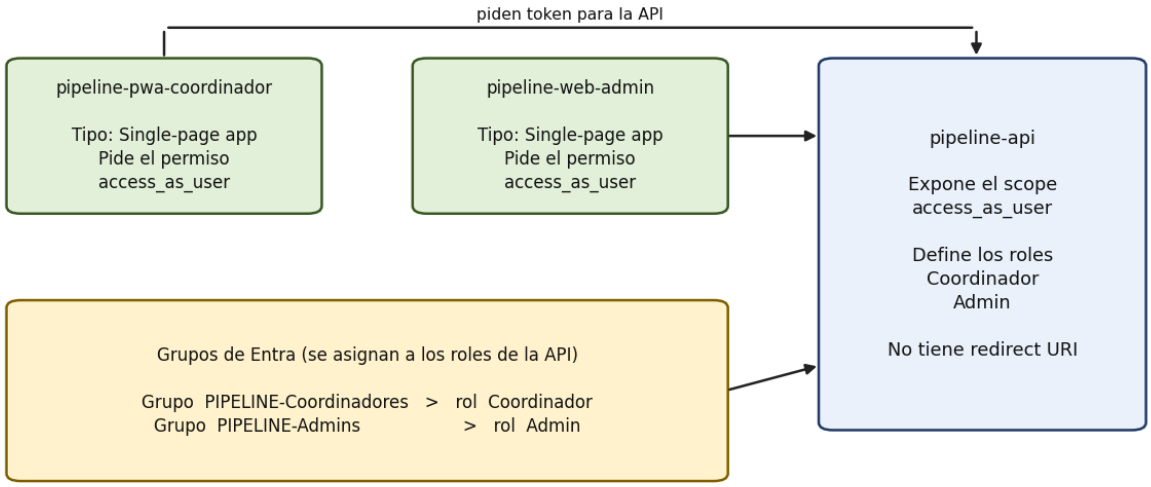


Figura 2. Los tres registros y los grupos de Entra.

Por qué dos frontends separados: cada uno tiene su propio registro, lo que permite aplicar políticas de acceso distintas al administrador (así quedó en el TDD).

3. Antes de empezar

3.1 Qué permisos necesitas

Para crear registros de aplicación en Entra necesitas, como mínimo, un rol como **Cloud Application Administrator** (la guía de Microsoft para registrar APIs lo indica así). El quickstart de SPA menciona también Application Administrator o Application Developer. Si no tienes ninguno, no pasa nada: la sección 8 incluye un mensaje listo para pedirlo al equipo de identidad.

3.2 Herramientas locales

- Node.js (versión LTS) y npm, para React.
- Python 3.12 (o el que uses) y pip, para FastAPI.
- Un editor como VS Code.

3.3 Hoja de datos a anotar

Durante la Parte A vas a copiar estos valores. Imprime esta tabla o cópiala a un archivo seguro. **Ningún valor de esta tabla es un secreto**, pero tu API y tus frontends no funcionan sin ellos.

Dato	Dónde se obtiene	Tu valor
Tenant ID	Overview de cualquier registro (Directory (tenant) ID)	
API Client ID	Overview de pipeline-api (Application (client) ID)	
Application ID URI	Expose an API de pipeline-api. Normalmente api://<API Client ID>	
Scope completo	api://<API Client ID>/access_as_user	
Client ID del SPA coordinador	Overview de pipeline-pwa-coordinador	
Client ID del SPA admin	Overview de pipeline-web-admin	
Grupo de coordinadores	Nombre sugerido: PIPELINE-Coordinadores	
Grupo de administradores	Nombre sugerido: PIPELINE-Admins	

Nombres sugeridos. Los nombres pipeline-api, pipeline-pwa-coordinador y pipeline-web-admin son una propuesta. Cambia lo que quieras, pero usa el mismo nombre en todo tu equipo para no confundirte después.

4. Parte A · Configuración en Entra

Entra al centro de administración de Microsoft Entra (**entra.microsoft.com**) con tu cuenta corporativa. Si tienes acceso a varios tenants, confirma arriba a la derecha que estás en el de Tiendas 3B. Sigue los pasos en orden: cada uno depende del anterior.

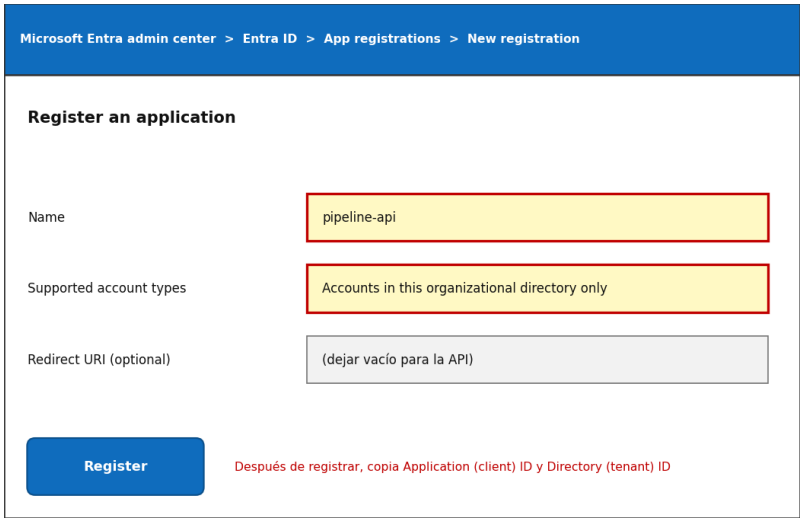
A1 Registrar la API

Dónde: Entra ID > App registrations > New registration

Qué hacer:

- **Name:** pipeline-api
- **Supported account types:** Accounts in this organizational directory only (un solo tenant, solo cuentas corporativas).
- **Redirect URI:** déjalo vacío. La API no inicia sesión de usuarios, solo recibe tokens.
- Haz clic en **Register**.
- Copia **Application (client) ID** y **Directory (tenant) ID** a tu hoja de datos (sección 3.3).

Resultado esperado: Ves la página Overview del registro con los dos identificadores.



Esquema ilustrativo (no es una captura real).

A2 Agregarte como propietario de la API

Dónde: App registrations > pipeline-api > Owners > Add owners

Qué hacer:

- Busca tu usuario, selecciónalo y confirma con **Select**.
- Hazlo también después en los dos registros de los frontends (pasos A6 y A7).

Resultado esperado: Tu nombre aparece en la lista de propietarios. Microsoft indica que, si no hay un propietario asignado tanto en la API como en la app que pedirá el permiso, la API no aparece al solicitar permisos.

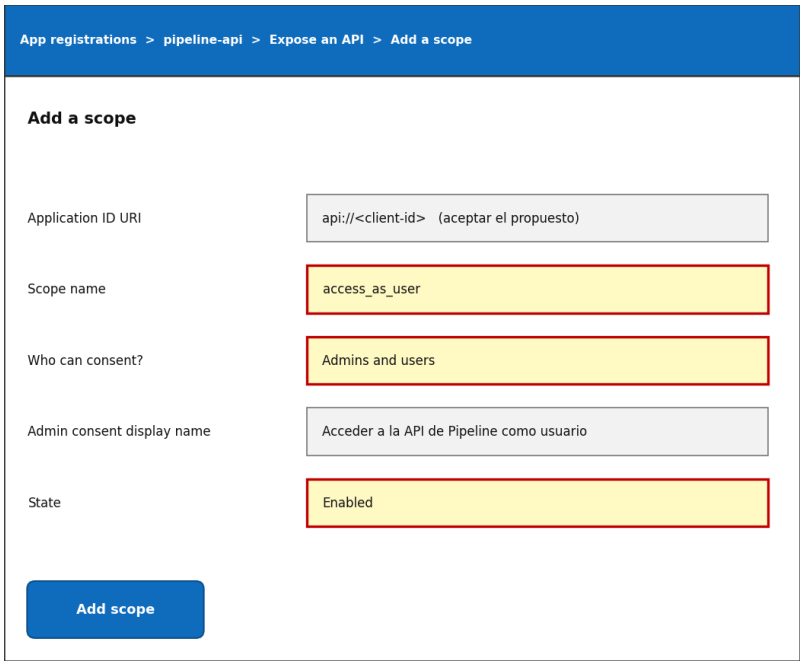
A3 Exponer la API y crear el scope access_as_user

Dónde: App registrations > pipeline-api > Expose an API

Qué hacer:

- Si Entra pide el **Application ID URI**, haz clic en **Add** y acepta el propuesto: **api://<API Client ID>**. Guárdalo.
- Haz clic en **Add a scope**.
- **Scope name:** access_as_user
- **Who can consent:** Admins and users
- Llena los textos de consentimiento (nombre y descripción para el administrador y para el usuario), por ejemplo «Acceder a la API de Pipeline como usuario».
- **State:** Enabled. Haz clic en **Add scope**.

Resultado esperado: El scope aparece en la lista con la forma api://<API Client ID>/access_as_user.



Esquema ilustrativo (no es una captura real).

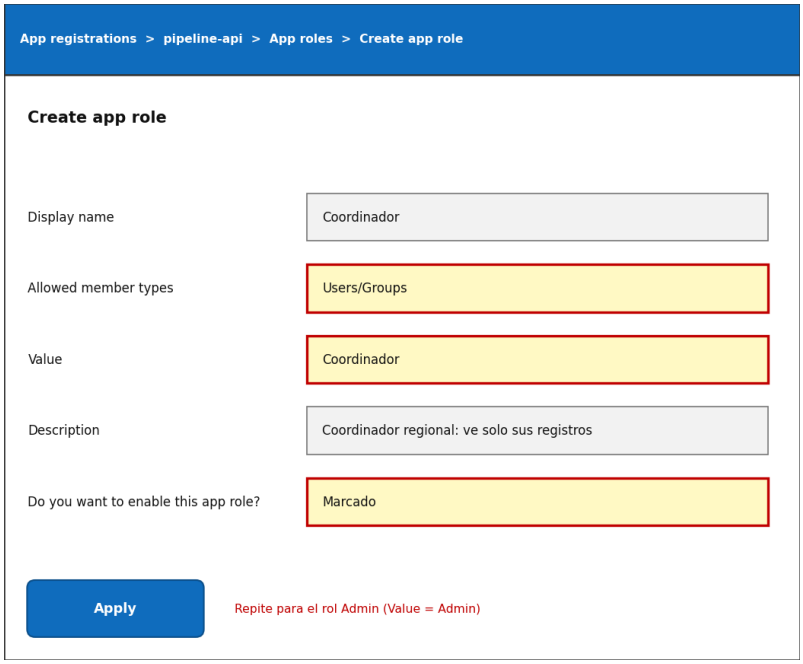
A4 Crear los roles Coordinador y Admin

Dónde: App registrations > pipeline-api > App roles > Create app role

Qué hacer:

- Primer rol: **Display name** = Coordinador. **Allowed member types** = Users/Groups. **Value** = Coordinador. Escribe una descripción. Deja marcado «enable this app role» y haz clic en **Apply**.
- Segundo rol: repite con **Display name** = Admin y **Value** = Admin.
- El campo **Value** es el texto que aparecerá dentro del token en la lista **roles**. Usa una sola palabra, sin espacios, y escríbela igual en tu código (las mayúsculas importan).

Resultado esperado: Dos roles en la lista, ambos habilitados.



Esquema ilustrativo (no es una captura real).

A5 Revisar la versión del token de la API (opcional pero recomendado)

Dónde: App registrations > pipeline-api > Manifest

Qué hacer:

- Busca la propiedad **requestedAccessTokenVersion** dentro del bloque **api** y déjala en **2**. Si tu portal solo muestra el manifiesto de AAD Graph, la propiedad se llama **accessTokenAcceptedVersion**.
- Guarda los cambios.
- Si no estás seguro de cómo quedó, continúa. El código de la Parte C acepta tokens v1 y v2. La v2 es la recomendada. Este punto **por validar** en tu portal: el nombre exacto de la propiedad depende de la pestaña del manifiesto que veas.

Resultado esperado: Al decodificar un token en la Parte D, el emisor (iss) termina en /v2.0.

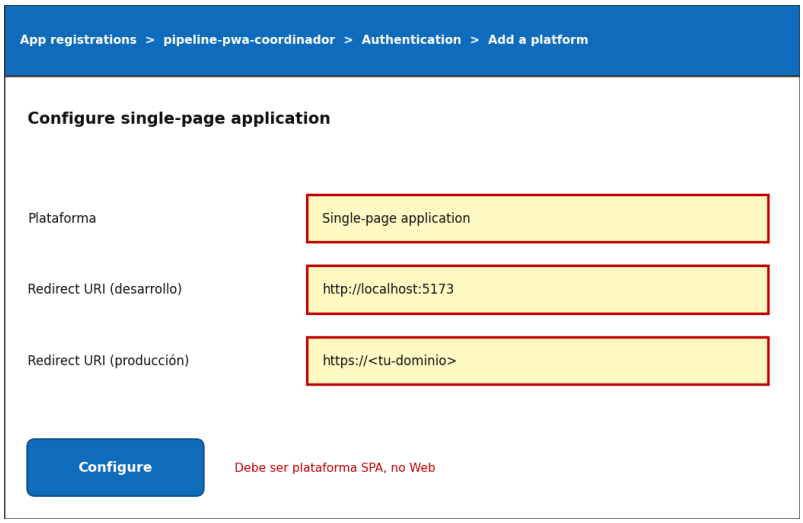
A6 Registrar el frontend del coordinador

Dónde: Entra ID > App registrations > New registration

Qué hacer:

- **Name:** pipeline-pwa-coordinador. **Supported account types:** solo este directorio (un solo tenant).
- **Redirect URI:** en el menú de plataforma elige **Single-page application (SPA)** y escribe **http://localhost:5173** (el puerto por defecto de Vite). Sin barra final.
- Haz clic en **Register** y copia el **Client ID** a tu hoja de datos.
- Después, en **Authentication**, agrega la URL de producción con **https** cuando la tengas. Todas deben estar bajo la plataforma SPA, no bajo «Web».
- En **Owners**, agrégate como propietario (paso A2).

Resultado esperado: En Authentication, la plataforma Single-page application muestra tus URI.



Esquema ilustrativo (no es una captura real).

A7 Registrar el frontend del administrador

Dónde: Entra ID > App registrations > New registration

Qué hacer:

- Repite A6 con **Name:** pipeline-web-admin y **Redirect URI** SPA = **http://localhost:5174**.
- Copia su **Client ID** a la hoja de datos y agrégate como propietario.

Resultado esperado: Dos registros SPA distintos, cada uno con su propio Client ID.

A8 Dar a cada frontend permiso para llamar a la API

Dónde: App registrations > (cada SPA) > API permissions > Add a permission

Qué hacer:

- Elige **APIs my organization uses** (o **My APIs**) y busca **pipeline-api**.
- Selecciona **Delegated permissions**, marca **access_as_user** y haz clic en **Add permissions**.
- Haz clic en **Grant admin consent for <tu organización>** (requiere un rol de administrador). Así los usuarios no ven una pantalla de consentimiento.
- Repite en los dos SPA.

Resultado esperado: El permiso access_as_user aparece con estado «Granted».

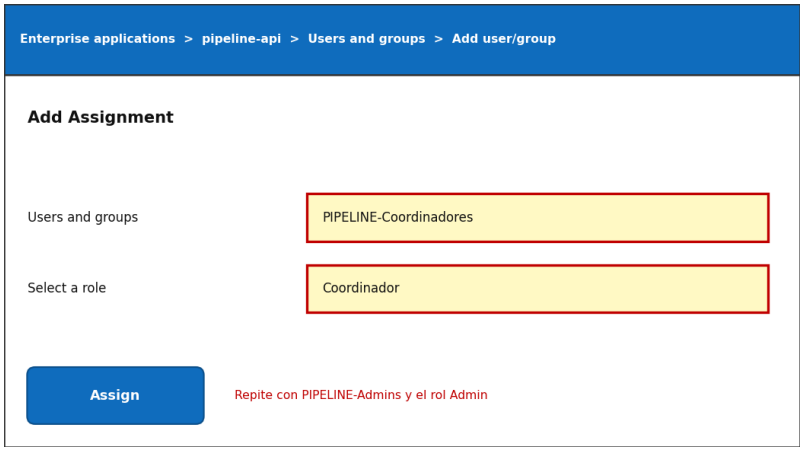
A9 Crear los grupos y asignarlos a los roles

Dónde: Entra ID > Groups > New group / Enterprise applications > pipeline-api > Users and groups

Qué hacer:

- Crea dos grupos de tipo **Security**: **PIPELINE-Coordinadores** y **PIPELINE-Admins**. Agrega a los usuarios como miembros directos.
- Ve a **Enterprise applications**, abre **pipeline-api** > **Users and groups** > **Add user/group**.
- En **Users and groups** elige PIPELINE-Coordinadores y en **Select a role** elige **Coordinador**. Haz clic en **Assign**.
- Repite con PIPELINE-Admins y el rol **Admin**.
- Dos advertencias: la asignación de roles a grupos puede requerir licencia Entra ID P1 o superior (confírmalo con identidad), y con grupos anidados la asignación puede no aplicar; usa membresía directa.

Resultado esperado: Cada grupo aparece asignado a su rol.



Esquema ilustrativo (no es una captura real).

A10 Exigir asignación para entrar

Dónde: Enterprise applications > pipeline-api > Properties

Qué hacer:

- **Assignment required?** = **Yes**. Guarda.
- Con esto, un usuario que no esté en ninguno de los dos grupos no recibe token para la API.

Resultado esperado: Un usuario sin grupo ve el error AADSTS50105 al intentar entrar.

5. Parte B · Frontend React con MSAL

Se usan los mismos pasos para la PWA del coordinador y para la web del administrador; solo cambia el Client ID y el puerto.

B1. Crear el proyecto

```
npm create vite@latest pwa-coordinador -- --template react-ts
cd pwa-coordinador
npm install
npm install @azure/msal-browser @azure/msal-react
```

B2. Variables de entorno

Crea el archivo **.env.local** en la raíz del proyecto. Vite solo expone variables que empiezan con VITE_.

```
VITE_TENANT_ID=<Tenant ID>
VITE_SPA_CLIENT_ID=<Client ID del SPA coordinador>
VITE_API_CLIENT_ID=<API Client ID>
VITE_API_URL=http://localhost:8000
```

B3. Configuración de MSAL

Archivo **src/auth/authConfig.ts**:

```
import { Configuration } from "@azure/msal-browser";

export const msalConfig: Configuration = {
  auth: {
    clientId: import.meta.env.VITE_SPA_CLIENT_ID,
    authority: `https://login.microsoftonline.com/${import.meta.env.VITE_TENANT_ID}`,
    redirectUri: window.location.origin,        // debe coincidir con el Redirect URI registrado
  },
  cache: {
    cacheLocation: "sessionStorage",             // ver nota sobre offline más abajo
  },
};

// El scope de TU API (no el de Microsoft Graph)
export const apiRequest = {
  scopes: [`api://${import.meta.env.VITE_API_CLIENT_ID}/access_as_user`],
};
```

Nota sobre offline: en una PWA offline-first quizá necesites **localStorage** para conservar la sesión entre cierres de la app. Es menos seguro que sessionStorage porque el token persiste en el dispositivo; decide con Seguridad, ya que hay datos personales de propietarios.

B4. Arrancar MSAL y envolver la app

Archivo **src/main.tsx**. MSAL debe inicializarse **antes** de pintar la app, y todo lo que use sesión va dentro de **MsalProvider**.

```
import React from "react";
import { createRoot } from "react-dom/client";
import { PublicClientApplication, EventType, AuthenticationResult } from "@azure/msal-browser";
import { MsalProvider } from "@azure/msal-react";
import { msalConfig } from "./auth/authConfig";
import App from "./App";

const pca = new PublicClientApplication(msalConfig);

pca.initialize().then(() => {
  const accounts = pca.getAllAccounts();
  if (accounts.length > 0) pca.setActiveAccount(accounts[0]);
```

```
  pca.addEventCallback((event) => {
    if (event.eventType === EventType.LOGIN_SUCCESS && event.payload) {
      pca.setActiveAccount((event.payload as AuthenticationResult).account);
    }
  });

  createRoot(document.getElementById("root")!).render(
    <React.StrictMode>
      <MsalProvider instance={pca}>
        <App />
      </MsalProvider>
    </React.StrictMode>
  );
});
```

B5. Pedir el token y llamar a la API

Archivo **src/api.ts**. Primero se intenta obtener el token en silencio; si Entra exige interacción (sesión vencida, cambio de política), se redirige al inicio de sesión.

```
import { IPublicClientApplication, InteractionRequiredAuthError } from "@azure/msal-browser";
import { apiRequest } from "./auth/authConfig";

async function getToken(pca: IPublicClientApplication): Promise<string> {
  const account = pca.getActiveAccount();
  if (!account) throw new Error("No hay sesión iniciada");
  try {
    const r = await pca.acquireTokenSilent({ ...apiRequest, account });
    return r.accessToken;
  } catch (e) {
    if (e instanceof InteractionRequiredAuthError) {
      await pca.acquireTokenRedirect({ ...apiRequest, account });  // sale de la página
    }
    throw e;
  }
}

export async function callApi(pca: IPublicClientApplication, path: string) {
  const token = await getToken(pca);
  const res = await fetch(`${import.meta.env.VITE_API_URL}${path}`, {
    headers: { Authorization: `Bearer ${token}` },
  });
  if (!res.ok) throw new Error(`HTTP ${res.status}`);
  return res.json();
}
```

B6. Pantalla mínima de prueba

Archivo **src/App.tsx**:

```
import { useState } from "react";
import { useIsAuthenticated, useMsal } from "@azure/msal-react";
import { apiRequest } from "./auth/authConfig";
import { callApi } from "./api";

export default function App() {
  const { instance } = useMsal();
  const isAuthenticated = useIsAuthenticated();
  const [data, setData] = useState<unknown>(null);
  const [error, setError] = useState("");

  if (!isAuthenticated) {
    return <button onClick={() => instance.loginRedirect(apiRequest)}>Iniciar sesión</button>;
  }
  return (
    <div>
      <p>Hola, {instance.getActiveAccount()?.name}</p>
```

```
      <button onClick={async () => {
        try { setData(await callApi(instance, "/me")); setError(""); }
        catch (e) { setError(String(e)); }
      }}>Probar /me</button>
      <button onClick={() => instance.logoutRedirect()}>Salir</button>
      {error && <p>{error}</p>}
      <pre>{JSON.stringify(data, null, 2)}</pre>
    </div>
  );
}
```

B7. Arrancar

```
npm run dev                     # coordinador, http://localhost:5173
npm run dev -- --port 5174      # en el proyecto del admin, http://localhost:5174
```

Dos reglas para evitar el error más común: (1) usa **loginRedirect**, no popup, en el teléfono; la documentación recomienda redirect cuando los popups están restringidos. (2) No llames **acquireTokenRedirect** mientras MSAL tenga una interacción en curso, o caerás en un bucle de redirecciones. Con los hooks de msal-react, revisa que **inProgress** sea InteractionStatus.None antes de pedir token automáticamente.

6. Parte C · Backend FastAPI

El backend hace tres cosas con cada petición: (1) toma el token del encabezado Authorization, (2) verifica firma, emisor, audiencia y expiración, y (3) revisa que el rol sea el permitido. El **oid** sale siempre del token, nunca de un parámetro que envíe el cliente.

C1. Instalar

```
pip install fastapi uvicorn "pyjwt[crypto]"
```

C2. Validación del token (auth.py)

```
import os
import jwt
from jwt import PyJWKClient
from fastapi import Depends, HTTPException
from fastapi.security import HTTPBearer

TENANT_ID = os.environ["ENTRA_TENANT_ID"]
API_CLIENT_ID = os.environ["ENTRA_API_CLIENT_ID"]

JWKS_URL = f"https://login.microsoftonline.com/{TENANT_ID}/discovery/v2.0/keys"
ISSUERS = [
    f"https://login.microsoftonline.com/{TENANT_ID}/v2.0",  # tokens v2 (recomendado)
    f"https://sts.windows.net/{TENANT_ID}/",                # tokens v1 (si el manifiesto sigue en v1)
]
AUDIENCES = [API_CLIENT_ID, f"api://{API_CLIENT_ID}"]

_jwks = PyJWKClient(JWKS_URL, cache_keys=True)
_bearer = HTTPBearer(auto_error=True)


def get_claims(cred=Depends(_bearer)) -> dict:
    token = cred.credentials
    try:
        signing_key = _jwks.get_signing_key_from_jwt(token).key
        return jwt.decode(
            token, signing_key,
            algorithms=["RS256"],
            audience=AUDIENCES,
            issuer=ISSUERS,
            options={"require": ["exp", "iss", "aud"]},
        )
    except jwt.PyJWTError:
        raise HTTPException(status_code=401, detail="Token inválido o expirado")


def require_role(*allowed: str):
    def checker(claims: dict = Depends(get_claims)) -> dict:
        if not set(claims.get("roles", [])) & set(allowed):
            raise HTTPException(status_code=403, detail="No tienes permiso para esta acción")
        return claims
    return checker
```

C3. Rutas de ejemplo (main.py)

```
from fastapi import Depends, FastAPI
from fastapi.middleware.cors import CORSMiddleware
from auth import require_role

app = FastAPI()

# Solo los orígenes de tus dos frontends (ajusta a tus URL reales)
app.add_middleware(
    CORSMiddleware,
    allow_origins=["http://localhost:5173", "http://localhost:5174"],
    allow_methods=["*"],
    allow_headers=["Authorization", "Content-Type"],
)

@app.get("/me")
def me(claims: dict = Depends(require_role("Coordinador", "Admin"))):
    return {"oid": claims["oid"], "roles": claims.get("roles", []), "nombre": claims.get("name")}

@app.get("/admin/ping")
def admin_ping(claims: dict = Depends(require_role("Admin"))):
    return {"ok": True}
```

C4. Arrancar la API

En macOS/Linux:

```
export ENTRA_TENANT_ID=<Tenant ID>
export ENTRA_API_CLIENT_ID=<API Client ID>
uvicorn main:app --reload --port 8000
```

En Windows (PowerShell):

```
$env:ENTRA_TENANT_ID="<Tenant ID>"
$env:ENTRA_API_CLIENT_ID="<API Client ID>"
uvicorn main:app --reload --port 8000
```

C5. Qué se probó de este código

Caso	Resultado esperado y obtenido
Sin encabezado Authorization	401
Token inválido	401
Token expirado	401
Audiencia que no es de tu API	401
Emisor que no es tu tenant	Rechazado (InvalidIssuer)
Token v2 con rol Coordinador en /me	200 y devuelve oid y roles
Token sin roles en /me	403
Rol Coordinador en /admin/ping	403
Rol Admin en /admin/ping	200

Las pruebas usaron tokens firmados con una llave local y un cliente de prueba de FastAPI; no se conectó a tu tenant. La validación contra Entra real ocurre en la Parte D.

C6. Cómo conecta esto con la seguridad por coordinador

En el resto de la API, usa siempre **claims["oid"]** para filtrar. Cuando conectes Azure SQL, ese mismo **oid** se fija en **SESSION_CONTEXT** al abrir cada conexión y Row-Level Security termina de aislar los registros, como define tu TDD.

7. Parte D · Probar que todo funciona

Haz las pruebas en este orden. Si una falla, no sigas: corrige primero esa.

#	Prueba	Resultado esperado	Si falla
1	Abre http://localhost:5173 y haz clic en **Iniciar sesión** con un usuario del grupo PIPELINE-Coordinadores.	Entra te pide credenciales (y MFA si aplica) y regresa a la app. Se ve «Hola, <tu nombre>».	Ver errores AADSTS en la sección 8.
2	Haz clic en **Probar /me**.	Aparece tu oid y roles: ["Coordinador"].	401: audiencia o emisor. 403: falta el rol. CORS: ver sección 8.
3	Decodifica el token (ver abajo) y revisa aud, iss, scp, roles y oid.	aud = tu API; scp = access_as_user; roles = ["Coordinador"].	Sin roles: usuario sin asignar, o el token se pidió con otro scope.
4	Con un usuario de PIPELINE-Coordinadores, llama a /admin/ping desde curl.	403.	Si da 200, revisa que el usuario no esté también en PIPELINE-Admins.
5	Con un usuario de PIPELINE-Admins en el SPA admin, llama a /admin/ping.	200.	Sin rol Admin: revisa la asignación del grupo en el paso A9.
6	Con un usuario sin grupo, intenta entrar.	Error AADSTS50105 (no asignado).	Si entra, revisa el paso A10.

Cómo ver el contenido de un token

- En el navegador, abre las herramientas de desarrollo (F12) > pestaña Network, selecciona la llamada a /me y copia el valor de Authorization después de «Bearer ».
- Pega el token en **jwt.ms**, el decodificador de Microsoft, y revisa las claims.
- **No pegues tokens reales de producción en sitios de terceros.** Un token vigente funciona como una llave temporal.

Prueba por línea de comandos (con el token copiado):

```
curl -H "Authorization: Bearer <token>" http://localhost:8000/me
curl -H "Authorization: Bearer <token>" http://localhost:8000/admin/ping
```

Pruebas con el teléfono (obligatorias antes de dar esto por bueno)

Estas pruebas no se pueden hacer desde una computadora y quedan **por validar** con un Android administrado por Intune:

- Inicio de sesión con redirect dentro de la PWA instalada y en el navegador.
- Renovación del token después de varias horas sin conexión. El refresh token de un SPA dura 24 horas, y pasado ese tiempo la renovación en silencio puede fallar y obligar a iniciar sesión otra vez.
- Que la cola offline conserve las operaciones pendientes y las reenvíe después de un nuevo inicio de sesión.
- Comportamiento con las políticas de acceso condicional que defina identidad.

8. Errores comunes

Síntoma	Causa probable	Solución
AADSTS50011: la dirección de respuesta no coincide	El Redirect URI registrado no es exactamente igual al que usa la app (puerto, http/https, barra final).	Registra **http://localhost:5173** tal cual, en la plataforma SPA.
AADSTS9002326: solo se permite el canje entre orígenes para clientes de tipo SPA	El Redirect URI se registró bajo la plataforma **Web**.	Muévelo a la plataforma Single-page application.
AADSTS65001: falta consentimiento	No se otorgó el consentimiento de administrador.	Paso A8: Grant admin consent.
AADSTS50105: el usuario no está asignado a un rol	«Assignment required» está en Yes y el usuario no pertenece a ningún grupo asignado.	Agrégalo al grupo correcto (A9).
La API devuelve 401	Audiencia o emisor distintos a los configurados, token expirado, o Client ID equivocado en las variables.	Decodifica el token y compara **aud** e **iss** con auth.py.
La API devuelve 403	El token no trae la claim **roles**.	Revisa que el usuario esté asignado a un rol y que el token se pidió con el scope api://.../access_as_user. Con scopes como el de Microsoft Graph, los roles aparecen en el ID token pero no en el de acceso.
Cambié los roles y el token no se actualiza	Los roles se escriben en el token cuando se emite.	Cierra sesión y vuelve a entrar.
Bucle infinito de redirecciones	Se llama a acquireTokenRedirect mientras hay una interacción en curso.	Solo pide token cuando inProgress sea None.
Error de CORS en el navegador	El origen del frontend no está en allow_origins.	Agrégalo en main.py (C3).
La API no aparece al agregar permisos	No hay propietario asignado en la API o en el SPA.	Paso A2.

Qué pedir al equipo de identidad

Si no tienes permisos para crear registros, puedes enviar este mensaje (ajusta nombres):

Hola, estamos construyendo el sistema de Pipeline de Posibles Tiendas (React + FastAPI) y necesitamos configurar la autenticación con Entra ID, de un solo tenant.

1) Tres registros de aplicación: **pipeline-api** (sin Redirect URI), **pipeline-pwa-coordinador** y **pipeline-web-admin** (plataforma SPA; URI de desarrollo http://localhost:5173 y http://localhost:5174, más las URL de producción cuando estén definidas).
2) En pipeline-api: Application ID URI api://<client-id>, scope delegado **access_as_user** (Admins and users) y app roles **Coordinador** y **Admin** (Users/Groups).
3) Permiso delegado access_as_user de pipeline-api en los dos SPA, con consentimiento de administrador.
4) Dos grupos de seguridad, **PIPELINE-Coordinadores** y **PIPELINE-Admins**, asignados a los roles Coordinador y Admin de pipeline-api, y «Assignment required» = Yes.
5) Confirmar si la licencia del tenant permite asignar roles de aplicación a grupos.
6) Por separado: política de acceso condicional (MFA y dispositivo administrado por Intune; alcance distinto para el administrador) y, más adelante, identidad administrada del backend y permisos de Microsoft Graph para notificaciones en Teams.

Necesito de vuelta: Tenant ID, Client ID de los tres registros y los nombres de los grupos. Gracias.

Fuentes

Documentación oficial de Microsoft Learn consultada para esta guía:

- Quickstart: Sign in users in a single-page app (SPA) — learn.microsoft.com/entra/identity-platform/quickstart-single-page-app-sign-in
- Add app roles and get them from a token — learn.microsoft.com/entra/identity-platform/howto-add-app-roles-in-apps
- Configure an application to expose a web API — learn.microsoft.com/entra/identity-platform/quickstart-configure-app-expose-web-apis
- Expose scopes in a protected web API — learn.microsoft.com/entra/identity-platform/scenario-protected-web-api-expose-scopes
- Single-page application: Acquire a token to call an API — learn.microsoft.com/entra/identity-platform/scenario-spa-acquire-token
- Get started with MSAL React — learn.microsoft.com/entra/msal/javascript/react/getting-started
- Respuestas de la comunidad (Stack Overflow) sobre la claim roles en el access token y sobre bucles de redirección con msal-react; son orientativas, no documentación oficial.

Los nombres exactos de menús, la licencia necesaria para asignar roles a grupos, la propiedad del manifiesto para la versión del token y el comportamiento en la PWA instalada son puntos por validar en el portal y en un dispositivo real.